

Interim Project Report: Political Bias Detection

Sergio Rojas-Aguilar

1. Project Summary

The goal of this project is to build an NLP pipeline that detects, quantizes and visualizes political bias in news media by analyzing how different articles portray the same political entities. Standard sentiment analysis often fails on news text because it scores the overall mood of a sentence rather than the author's stance toward a specific subject. By combining Named Entity Recognition with Aspect-Based Sentiment Analysis, this project aims to specific political figures and measures the targeted sentiment directed at them to computationally identify media bias.

2. Data

The project leverages three primary data sources to ensure both scale and ground-truth accuracy:

- **AllSides Media Bias Ratings:**
Used to reference for publisher orientation. The dataset provides bias ratings more than 1,400 news outlets, allowing me to select "opposing" articles for comparison.
- **BASIL (Bias Annotation Spans on the Informational Level):** A specialized dataset of 300 news articles organized into "triplets" (ie Left, Center, and Right perspectives). More than 1,700 manual annotations for lexical and informational bias.
- **MBIC (Media Bias Annotation Dataset):** A large-scale dataset of annotated articles used to test the model's ability to identify biased vs. neutral framing at the sentence level.

3. Approach

Multiple transformer pipeline:

- **Coreference Resolution:** The text is processed through `fastcoref` to link pronouns (ei, "he," "the President") back to their root entities, ensuring sentiment is correctly attributed.
- **NER & Entity Linking:** Using `spaCy` with "en_core_web_trf" for extraction. Entity linking maps various name iterations. Like "Trump" vs. "45th President" to connect identifiers for unified sentiment aggregation.
- **Aspect-Based Sentiment Analysis (ABSA):** Using a fine-tuned DeBERTa V3 model to calculate the sentiment score of the context window specifically surrounding the target entity.

Or

- **Stance Detection:** Instead of basic sentiment scoring, the pipeline uses approach with DeBERTa-v3 to determine if the author's stance. FAVOR, AGAINST, or NEUTRAL toward the specific entity.
- **Backend Infrastructure:** A Python backend handling the NLP inference, data, and API routing.

- **Frontend Visualization:** A React/TypeScript dashboard to visualize the data. Mapping entity stance.
- **Change 1: ABSA to Stance Detection.** Aspect-Based Sentiment Analysis captures the emotion of a sentence. Stance detection solves this by identifying if the text is "favor of or against" the target entity.
- **Change 2: C++ to Python Backend.** The original plan use a C++ Crow server and llama.cpp. Instead I'll use Python backend. The Python ecosystem is more native integration with Hugging Face, spaCy, and fastcoref, eliminating coding C++ overhead.
- **Change 3: Frontend Code Generation.** I am utilizing my existing knowledge of React and TypeScript alongside OpenAI Codex to build the visualization dashboard. This allows me to accelerate the UI development, focus my more hours on the NLP pipeline, and simultaneously submit the resulting full-stack project into an upcoming **AI creator challenge** from handshake.

4. Progress So Far

- **Dataset Integration:** reviewed the AllSides ratings and the BASIL dataset. The "triplet" structure of BASIL has been verified as a viable baseline for comparing entity sentiment across different ideological poles.
- **Pipeline Setup:** Established the coreference and NER sequence. More testing of data is required.
- **Challenges:** challenge is the computational cost of running coref-resolution and transformer-based ABSA in a single stream.

5. Next Steps

- **Infrastructure:** Finalize the scoring logic for the Zero-Shot NLI Stance model on the BASIL dataset and validate against the manual "bias spans" provided in the dataset.
- **Visualization Step:** Create agentic markdown structure for **Codix** to use D3.js or Chart.js to map entity sentiment ideological axis based on the AllSides ratings.
- **Integration:** Wrap the pipeline in a FastAPI or Flask endpoint to API backend to handle Python.

6. Questions or Concerns

Feedback Request: What I have been seeing is that overall more stance is neutral if averaged all sentences stance. Even with heavy biases source, it appears to have more factual statements than stance statements. 5 factual statements and 2 or 3 clear stance bias per entity. How should I handle that?